Design Document

Capacitas

Aman Messinezis

Version 3

Contents

Introduction

The aim of this system is to create a system that automates the process of tracking monthly Azure costs to calculate what clients need to pay the Company and send emails to colleagues who will be using it. At the moment, Capacitas manually look over their server costs at the end of the billing period and perform calculations to see what clients should be paying for. This process is done on Azure Portal by looking at each resource group per client and the Cost Analysis in Cost Management. The system is to be used by project managers who need to track Azure costs for their Azure virtual machines so that they know what clients need to pay for the virtual machines they use every month.

Python will be the programming language used to create this automation. This is because Python comes with several libraries associated with visualisation. As the primary purpose of the system is to track costs over time, suitable visualisations would be required to provide the best interface for the clients by providing well-suited charts that can be easily interpreted. The need for an Azure API is essential to connect to Azure servers and extract the necessary information from the Company's resources hosted on Azure.

The problem with the current system is that it's highly prone to human error. With several data sets across different clients, there's always the possibility of colleagues handling costs on Azure to make genuine errors. With this, opens the problem of overcutting or undercutting client costs.

Another problem with the current system is that it's a very time-consuming task. Because there are so many clients with different subscriptions and resource groups within those descriptions, it's a tiresome job for colleagues to have to work out how much has been spent by their clients, especially when this must be done every month. This is a problem, as

colleagues could spend more time on their primary role as a consultant and take on more roles that can't be automated, such as some analysis.

Proposed solution

The system enables users to sign in to their account, which, when successful, consists of their saved queries that they may want to run immediately or sent to them, usually at the end of every billing month. This system is to display the VM costs spent for client ands a visualisation to see how this has changed over time. This is an optional view that can be added by the user. This will heavily reduce the amount of time project managers need to spend on managing client costs and virtually eliminate the likelihood of any human error.
The solution will require the need of an Azure API. This is so the application can connect to the Azure servers and extract all necessary resource cost data based on what client the user is querying information for. The view will show the total cost used by the given client and a breakdown of this cost across all the cloud resources used.
This application will be run at times requested by the user, and an email is sent to them displaying all the necessary information regarding cloud costs used for a given client. Due to this, an SMTP protocol will be imported to send emails to users. As this will have to be run at different times, depending on when the user requests times when emails should be sent to them, this application will have to be hosted on an in-house server to be sure to run at given times.
Furthermore, the need for a server is essential to store user details so that when a user does attempt to log in on the front end of the application, then an API request is made to the back end of the server to double-check details and see whether they are correct or not, based on the user details stored on the server database, using an SQL query.
A command prompt window will be used to display information if the user uses the application. However, there may be times when a pop-up window is required for the need of clickable buttons.

Summary

The following document goes over the dynamic view of the system and goes into further depth with the existing diagrams made in the Requirement Specification Document. More specifically, refining the Analysis Class Diagram into its Design Class counterpart, including adding a GUI class that comes in handy when designing the Activity Diagram for the application'.

I have created sequence diagrams for the more critical use cases and described how classes are to communicate with each other over the use of class methods. All sequence diagram titles explicitly match their use case counterpart.

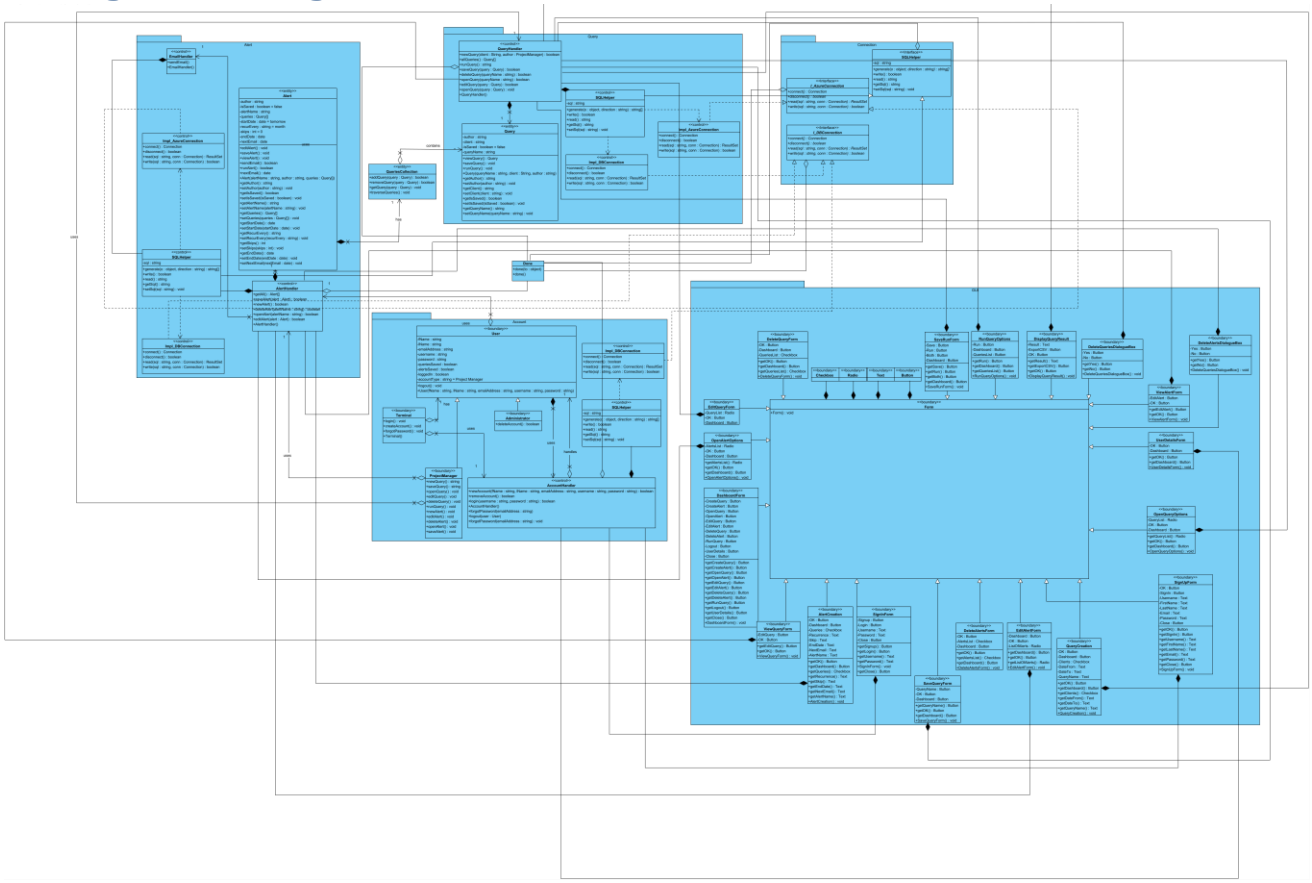
State machines have allowed me to be familiar with the states of particular classes based on external and internal factors.

The use of an Activity Diagram has given me a clear understanding of not only how my application will exist to the end-user at an abstract level but also how the user will navigate from page to page via different button clicks.

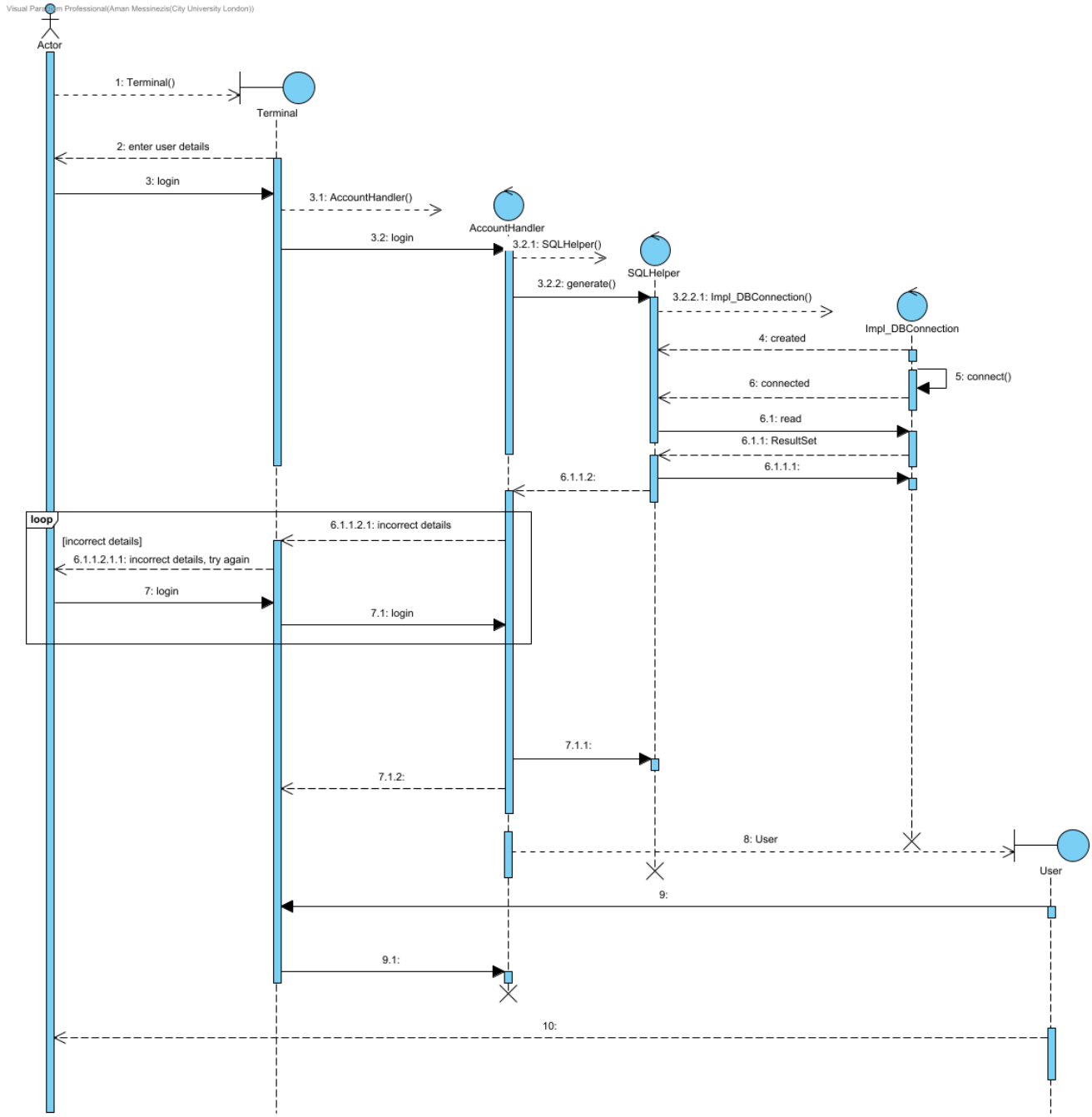
I've implemented an Entity Relationship Diagram that later became a relational database using the SQLite dialect to store necessary persistent data such as user details and saved queries. The representative set of operational SQL statements is attached to this submission, including the DDL and DML statements necessary for achieving persistence of the main problem domain entity classes. There is also a Data Specification in the Visual Paradigm file that contains slightly more information about each entity and their attributes.

I have attached the Visual Paradigm document to this submission, should you need to view any diagrams necessary in a higher quality.

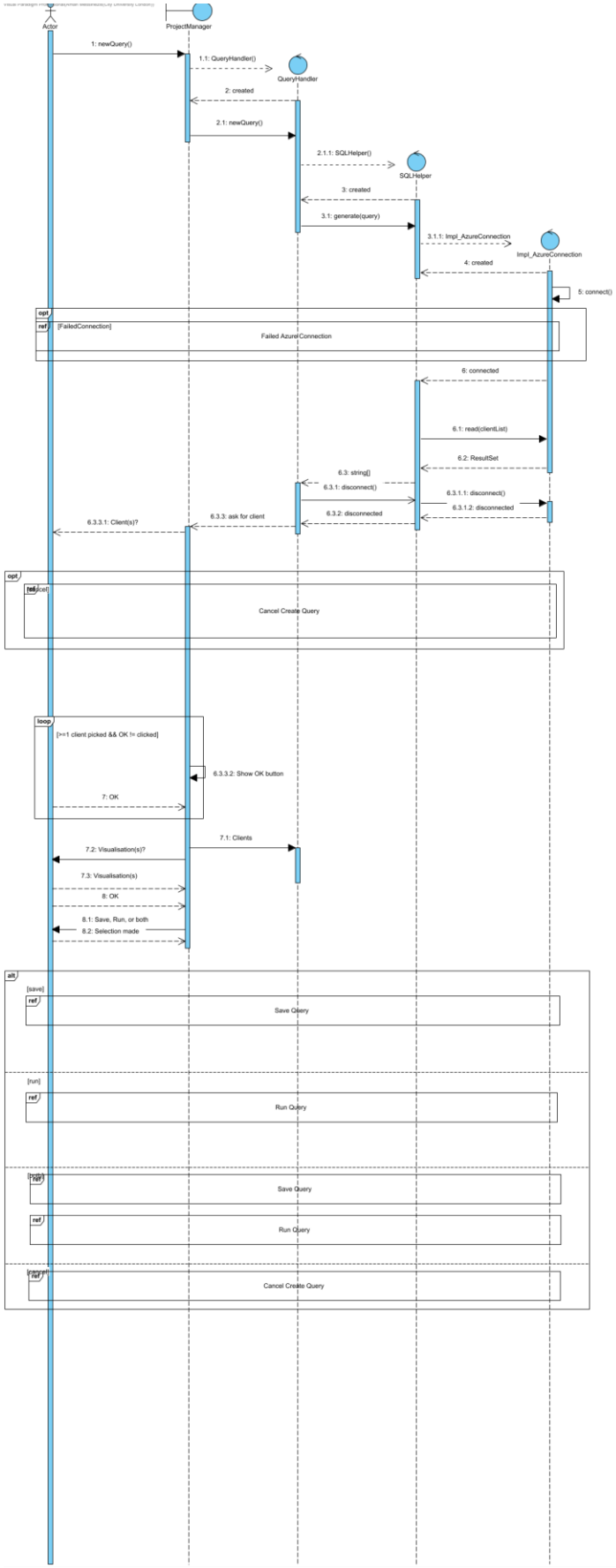
1. Design Class Diagram



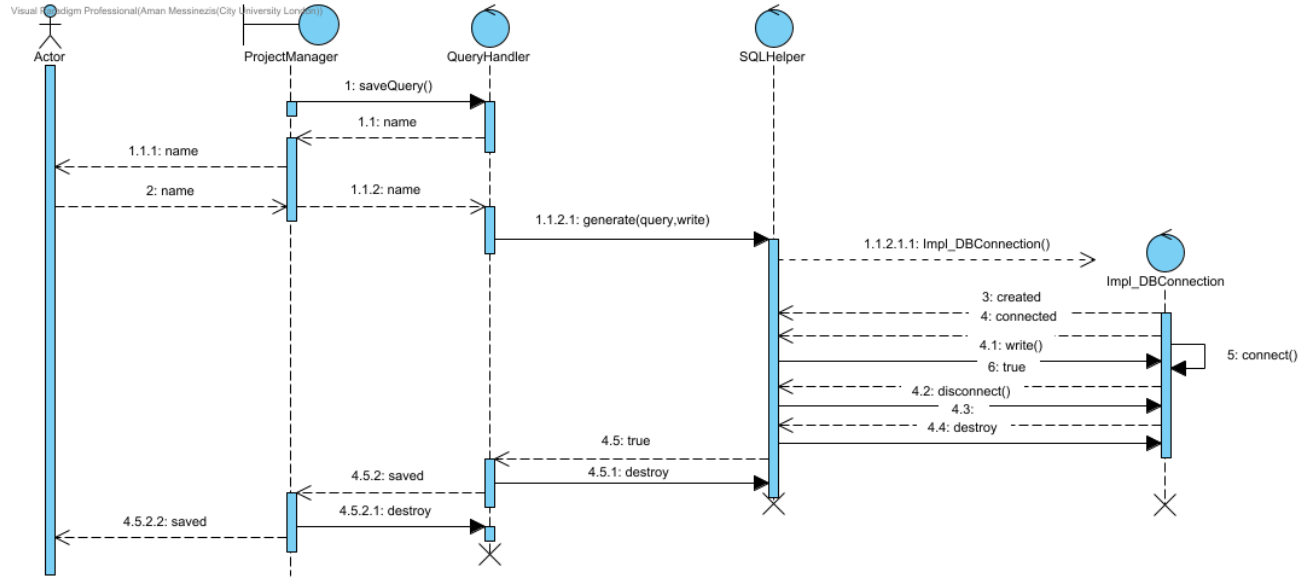
2. Log In


Visual Paradigm Professional(Aman Messinezis(City University London))
Actor
1: Terminal()
Terminal
2: enter user details
3: login
3.1: AccountHandler()
AccountHandler
3.2: login
3.2.1: SQLHelper()
SQLHelper
3.2.2: generate()
3.2.2.1: Impl_DBConnection()
Impl_DBConnection
4: created
5: connect()
6: connected
6.1: read
6.1.1: ResultSet
6.1.1.1:
6.1.1.2:
loop
6.1.1.2.1: incorrect details
[incorrect details]
6.1.1.2.1.1: incorrect details, try again
7: login
7.1: login
7.1.1:
7.1.2:
8: User
User
9:
9.1:
10:

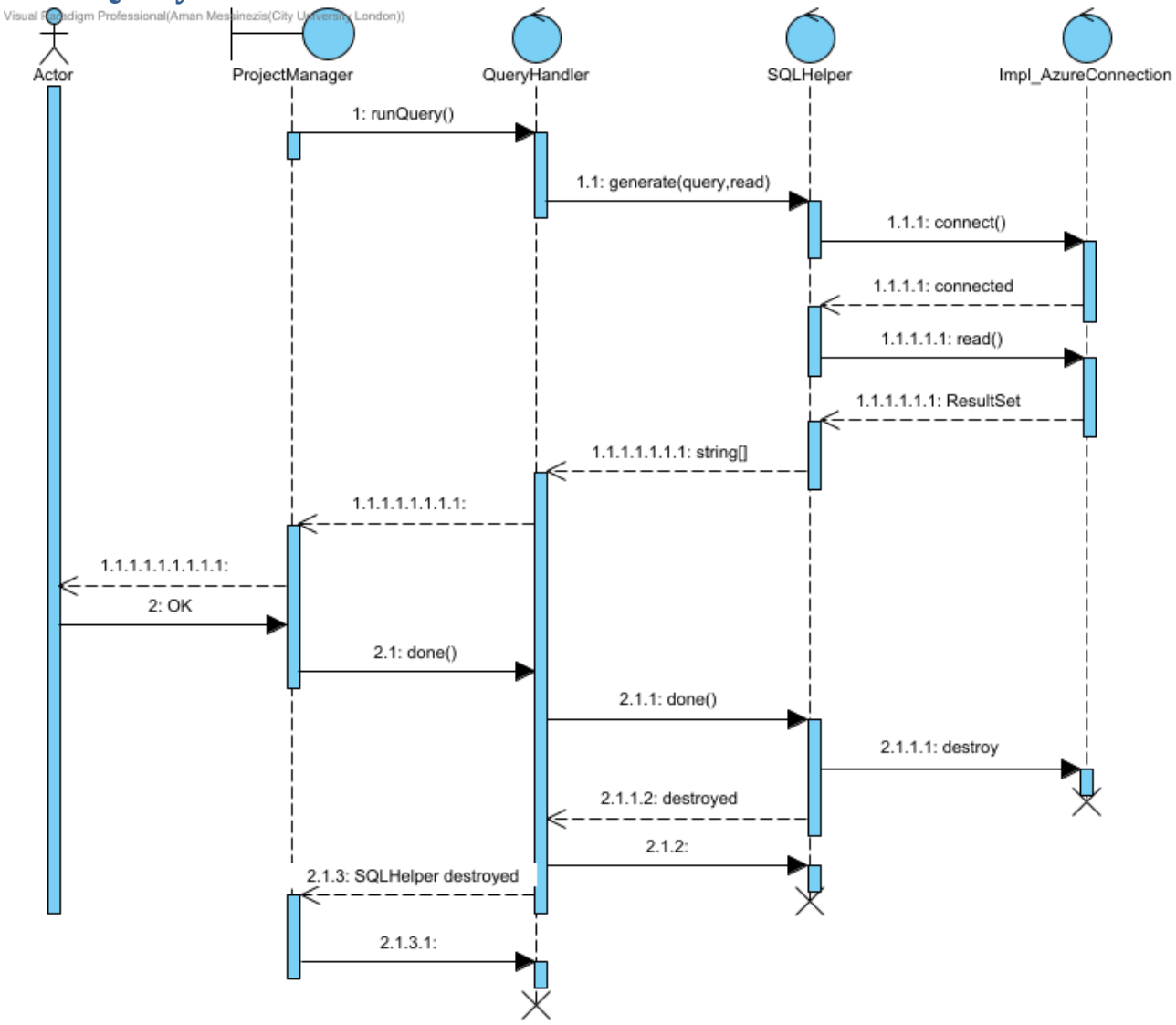
3. Create Query



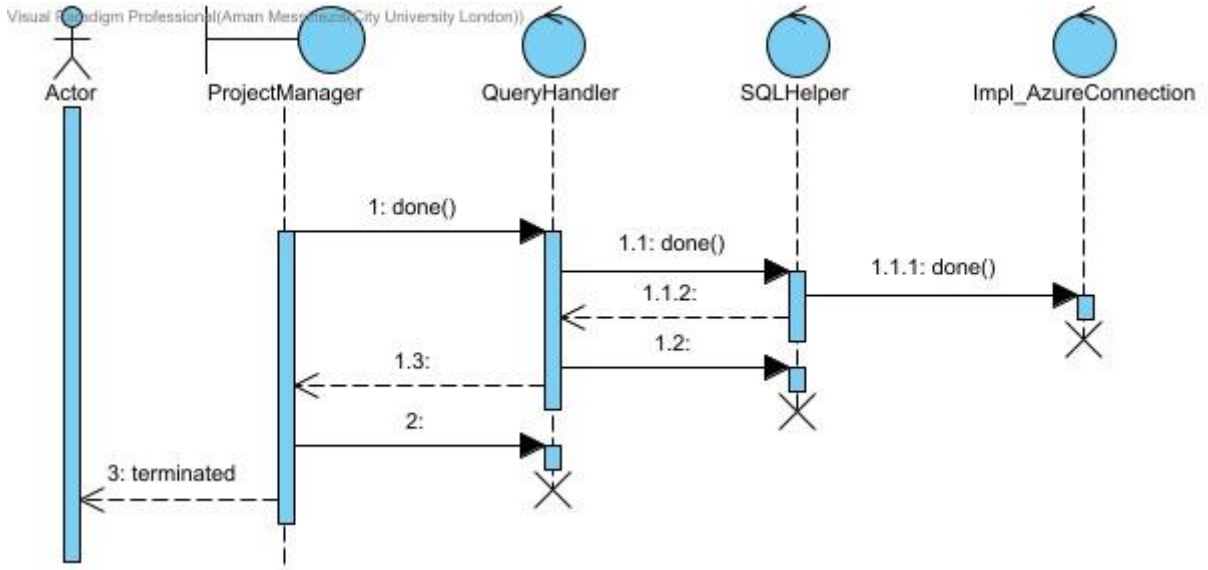
4. Save Query



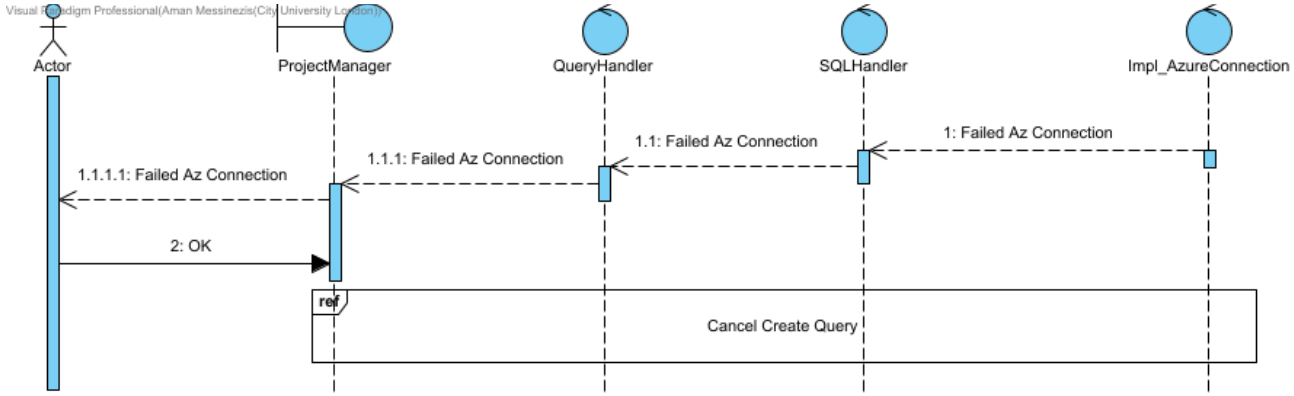
5. Run Query



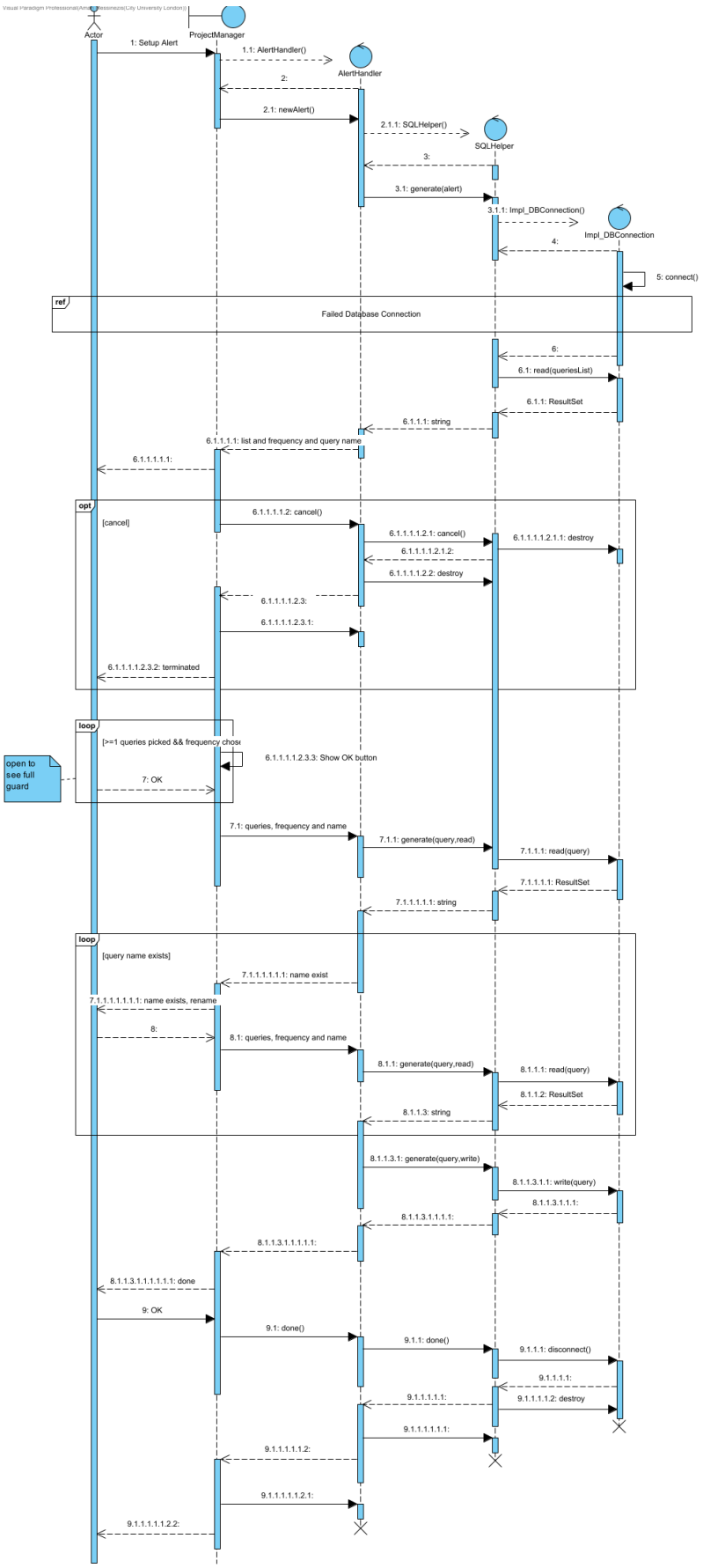
6. Cancel Create Query



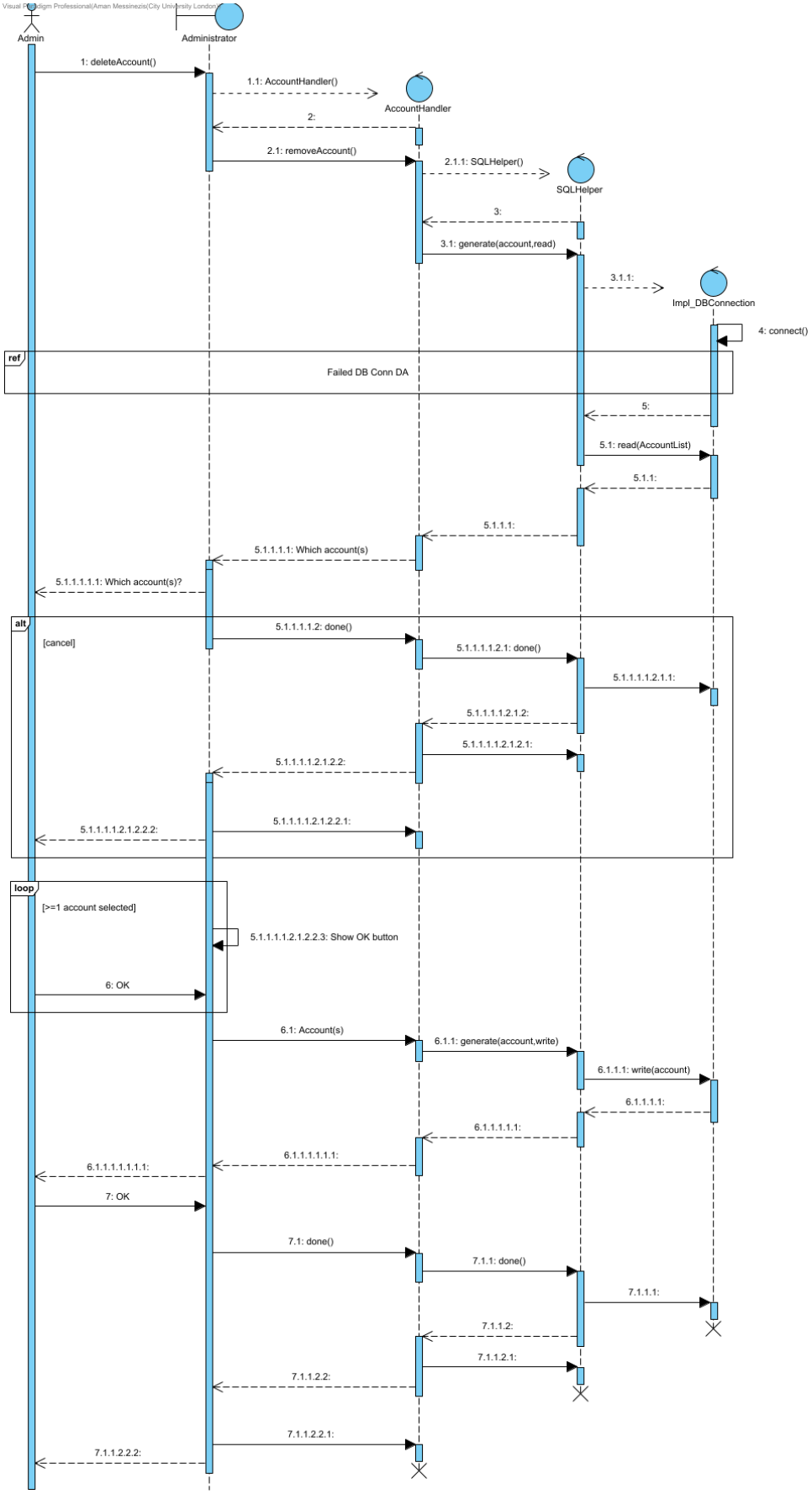
7. Failed Azure Connection



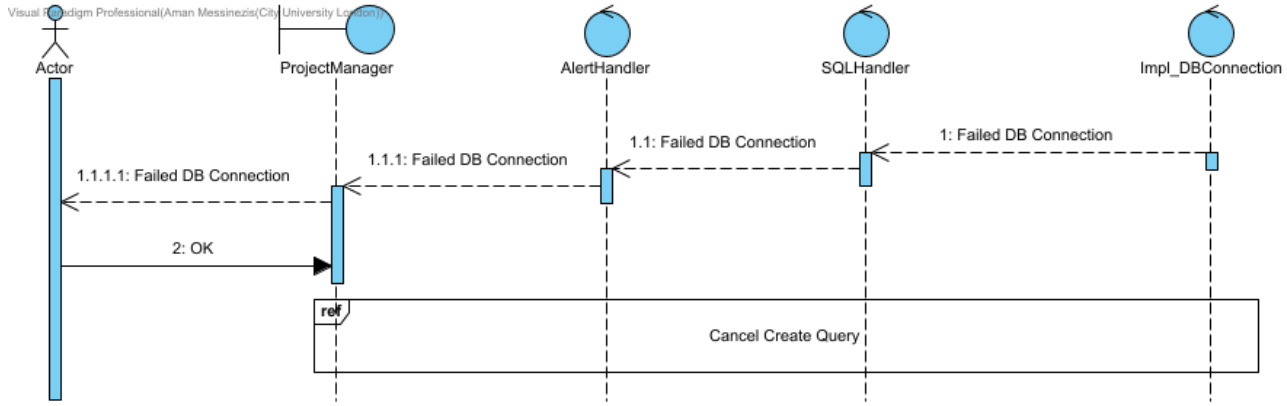
8. Setup Alert



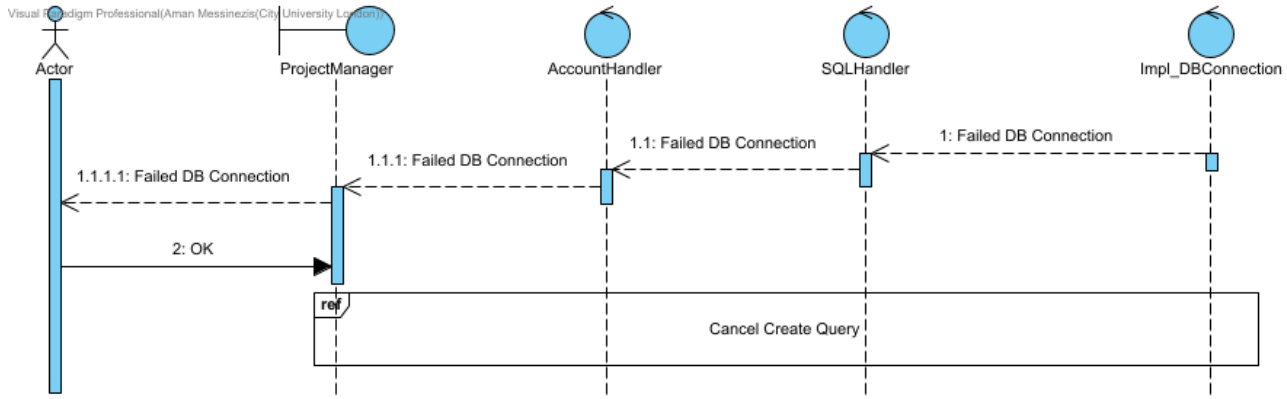
9. Delete Account



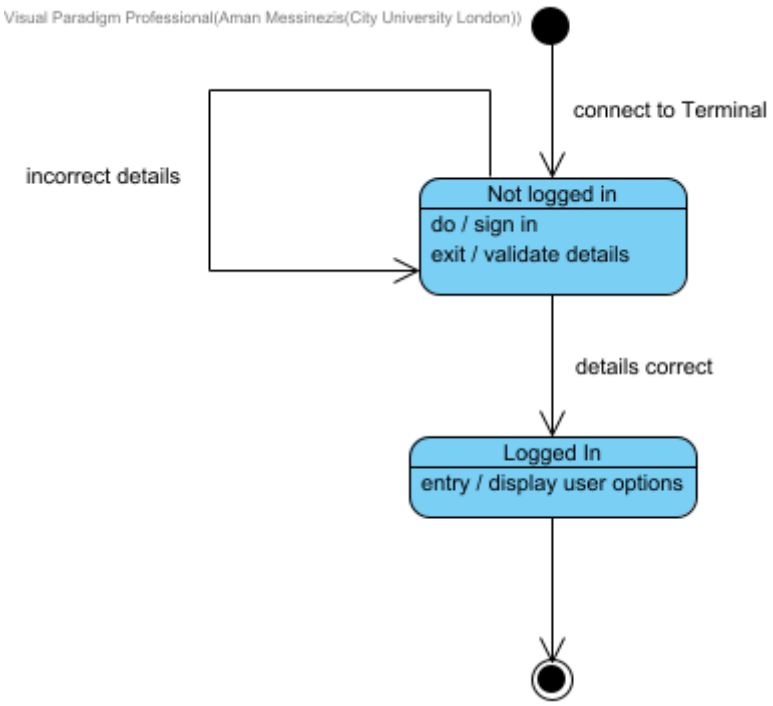
10. Failed Database Connection



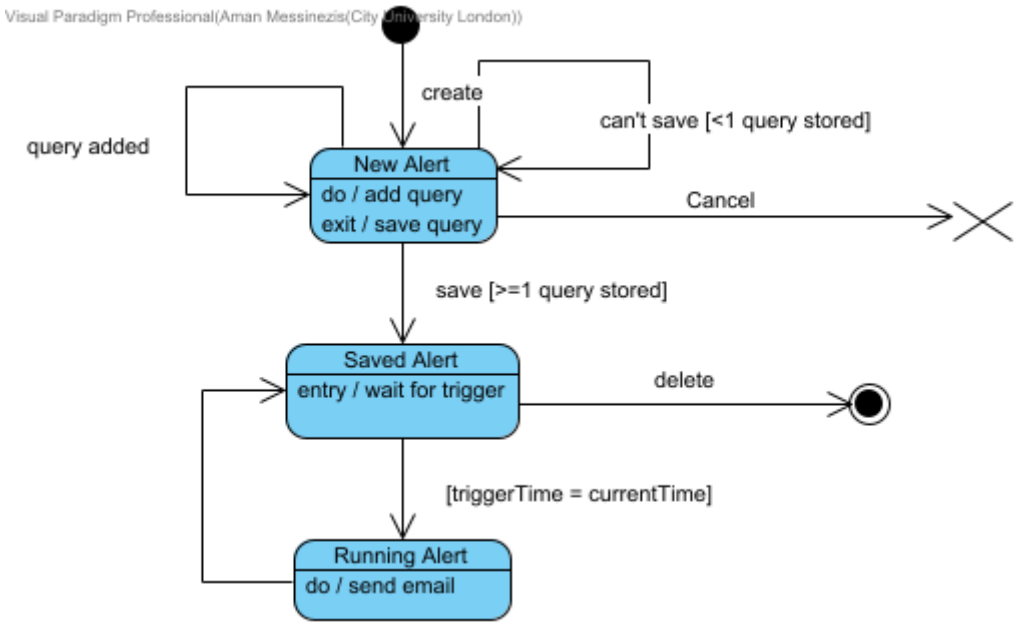
11. Failed DB Conn DA



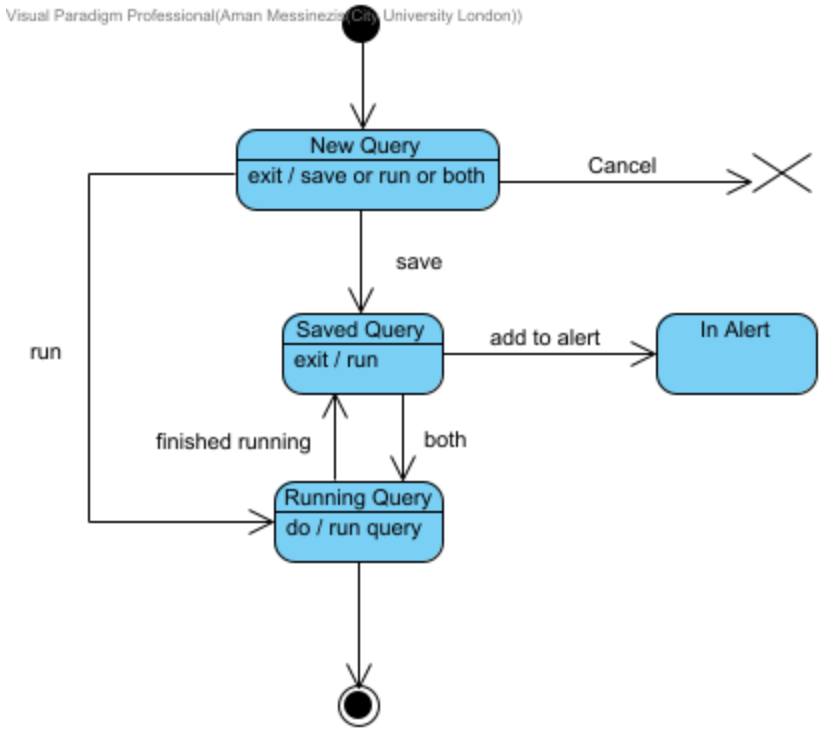
12. Log In



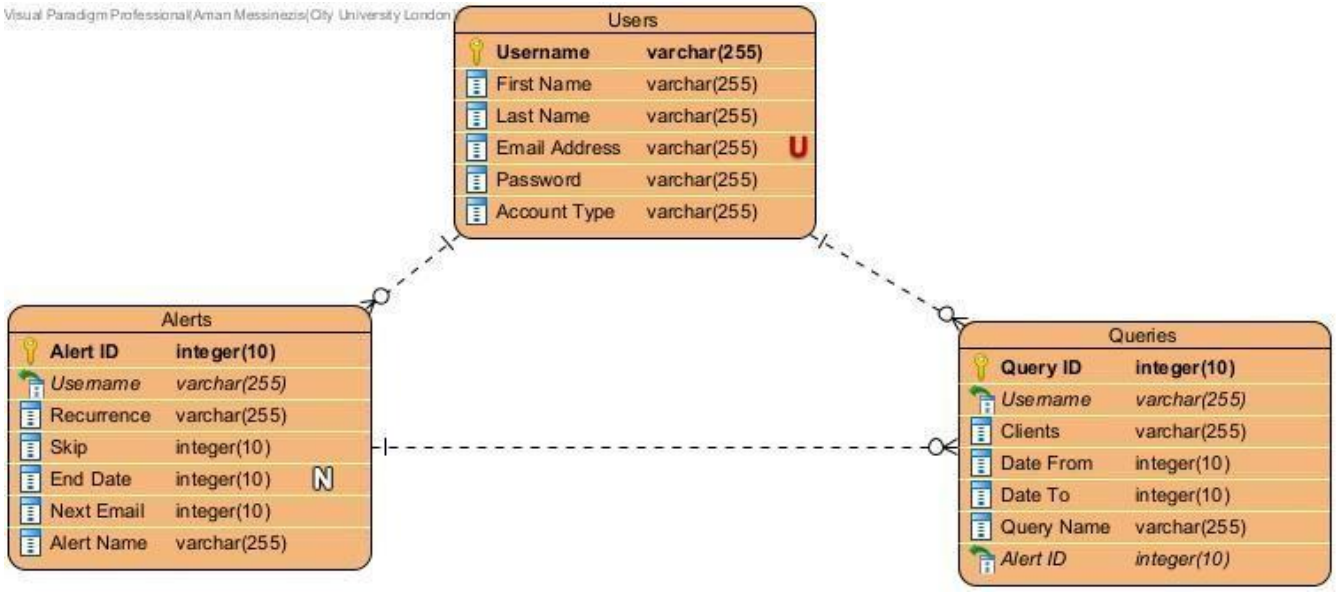
13. Alert



14. Query



15. Entity Relationship Diagram



I have purposely used the integer datatype for dates only to make searching easier.

The following SQL statements all use the SQLite Dialect.

```
CREATE TABLE Users (Username varchar(255) NOT NULL, "First Name" varchar(255) NOT
NULL, "Last Name" varchar(255) NOT NULL, "Email Address" varchar(255) NOT NULL
UNIQUE, Password varchar(255) NOT NULL, "Account Type" varchar(255) NOT NULL,
PRIMARY KEY (Username));

CREATE TABLE Alerts ("Alert ID" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
Username varchar(255) NOT NULL, Recurrence varchar(255) NOT NULL, Skip integer(10)
NOT NULL, "End Date" date, "Next Email" date NOT NULL, "Alert Name" varchar(255)
NOT NULL, FOREIGN KEY(Username) REFERENCES Users(Username));
CREATE TABLE Query ("Query ID" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT, Username
varchar(255) NOT NULL, Clients varchar(255) NOT NULL, "Date From" date NOT NULL,
"Date To" date NOT NULL, "Query Name" varchar(255) NOT NULL, "Alert ID" integer(10)
NOT NULL, FOREIGN KEY(Username) REFERENCES Users(Username), FOREIGN KEY("Alert ID")
REFERENCES Alerts("Alert ID"));
```

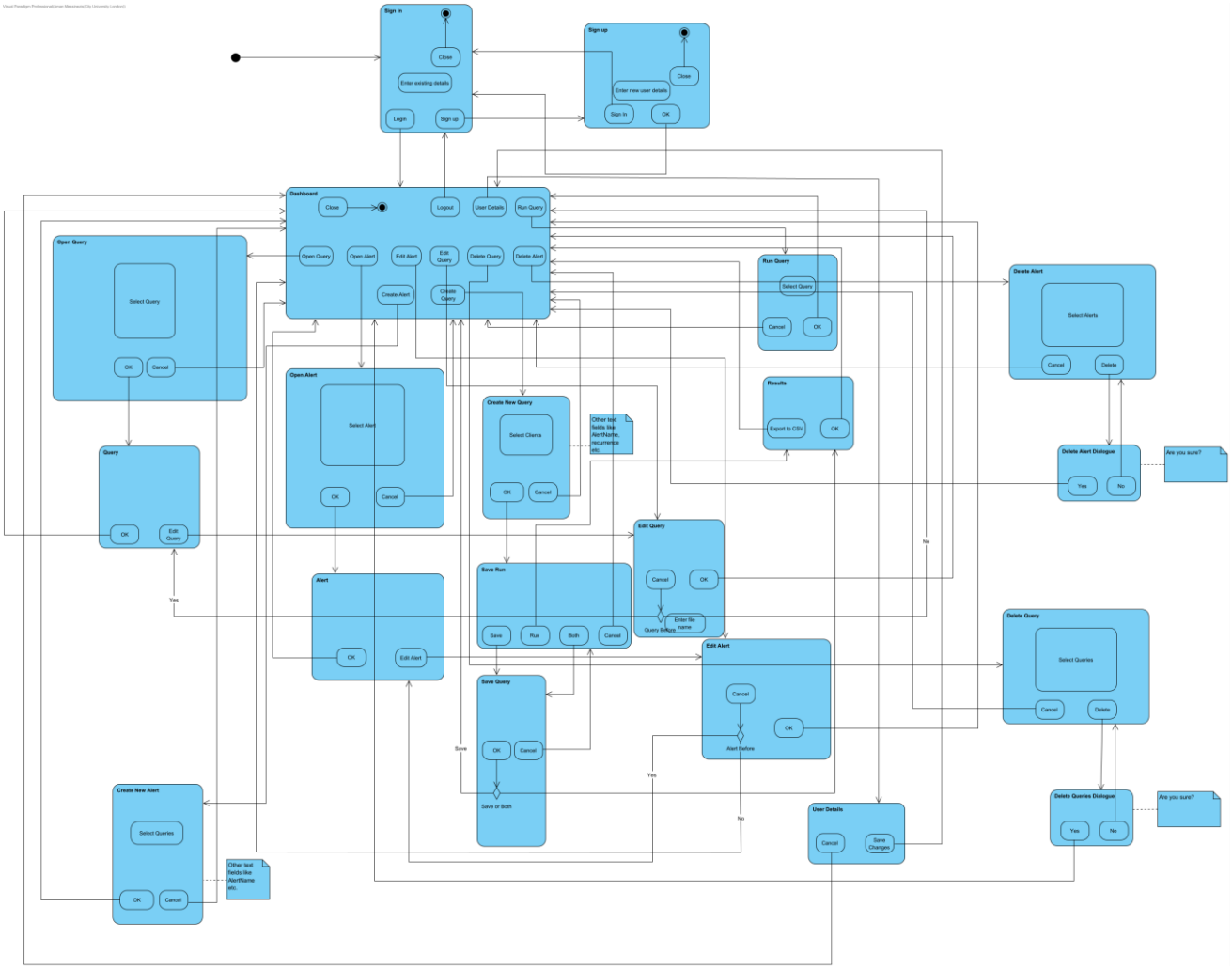
```
SELECT Password FROM "Users" WHERE Username = 'Aman';
SELECT "Query Name" FROM "Queries" WHERE Username = 'Aman' AND "Query Name" =
'MyQuery';
SELECT "Query Name" FROM "Queries" WHERE "Alert ID" = 1
```

```
INSERT INTO Users(Username, "First Name", "Last Name", "Email Address", Password,
"Account Type") VALUES ('Aman', 'Aman', 'Messinezis',
'amanmessinezis@capacitas.co.uk', 'SunnyDays', 'Consultant');
INSERT INTO Queries("Query ID", Username, Clients, "Date From", "Date To", "Query
Name", "Alert ID") VALUES (1, 'Aman', 'Deloitte', 20200501, 20200601,
'DeloitteMayJune2020', null);
INSERT INTO Alerts("Alert ID", Username, Recurrence, Skip, "End Date", "Next
Email", "Alert Name") VALUES (1, 'Aman', 'Month', 0, 20230101, 20220508,
'MyMonthlyAlert');
```

```
UPDATE Queries SET "Query Name" = 'MainQuery' WHERE "Query ID" = 1;
UPDATE Alerts SET Recurrence = 'Weekly' WHERE "Alert ID" = 1;
UPDATE Queries SET "Alert ID" = 1 WHERE "Query ID" = 1;
DELETE FROM Alerts WHERE "Alert ID" = 1;
DELETE FROM Queries WHERE "Query ID" = 1;
DELETE FROM Users WHERE Username = 'Aman';
```

All available attached in the submission portal

16. Activity Diagram



Appendix

To clarify, there were no changes made to the use case diagram since the Requirements Specification Document, which is why there is no “History of Document” section. Please view the Requirements Specification Document to view Use Case Descriptions